

Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Aula 7

Implementação da Análise Sintática

Análise sintática descendente

- Nos algoritmos de análise descendente, a construção começa pela raiz e segue na direção das folhas
- Quando todas as folhas têm símbolos terminais como valores, então a fronteira da árvore deve coincidir com a cadeia dada.

Análise sintática descendente

- Exemplo:
 - $E ::= T + E \mid T$
 - $T ::= F * T \mid F$
 - $F ::= a \mid b \mid (E)$
 - **w: a*b**

Análise sintática descendente

- Exemplo:

- $E ::= T + E \mid T$
- $T ::= F * T \mid F$
- $F ::= a \mid b \mid (E)$
- **w: a*b**

Ver “Exemplo1_Aula7.pdf”

Análise sintática descendente com retrocesso

- **Dificuldades associadas ao método**
 - Implementação complicada
 - Método ineficiente
 - Número de operações para analisar uma cadeia de comprimento n é uma função exponencial de n
 - Problema de gerenciamento de tabela de símbolos e código gerado

Análise sintática descendente com retrocesso

- **Eliminando o retrocesso**

- A maneira mais simples de evitar retrocesso é fazer com que o algoritmo sempre opte pela produção correta
 - Ou seja, acertar na primeira tentativa

Análise sintática descendente com retrocesso

- **Eliminando o retrocesso**

- Isto pode ser obtido para a classe de gramáticas que tem as seguintes restrições:

- As produções são da forma $A \rightarrow XA$, onde $X \in V_T$ e $A \in V(V_N \cup V_T)$

- Se $A \rightarrow X_1 A_1 | X_2 A_2 | \dots | X_n A_n$, com $X_i \in V_T$

- **Então X_i são todos distintos.**

Análise sintática descendente com retrocesso

- Exemplo:

- $E ::= a \mid b \mid +EE \mid *EE$

- Dada a cadeia $w = +a*ba$

$+a*ba$
 E
 $\uparrow$

(1)

$a*ba$
 E
 $\swarrow \downarrow \searrow$
 $+ \quad E \quad E$
 $\uparrow$

(2)

$*ba$
 E
 $\swarrow \downarrow \searrow$
 $+ \quad E \quad E$
 $\downarrow \uparrow$
 a

(3)

ba
 E
 $\swarrow \downarrow \searrow$
 $+ \quad E \quad E$
 $\downarrow \swarrow \downarrow \searrow$
 $a \quad * \quad E \quad E$
 $\uparrow$

(4)

a
 E
 $\swarrow \downarrow \searrow$
 $+ \quad E \quad E$
 $\downarrow \swarrow \downarrow \searrow$
 $a \quad * \quad E \quad E$
 $\downarrow \uparrow$
 b

(5)

λ
 E
 $\swarrow \downarrow \searrow$
 $+ \quad E \quad E$
 $\downarrow \swarrow \downarrow \searrow$
 $a \quad * \quad E \quad E$
 $\downarrow \downarrow$
 $b \quad a$

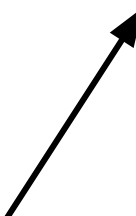
(6)

Análise sintática descendente com retrocesso

- As restrições impostas são muito severas para serem viáveis na prática
- Podemos generalizar a ideia obtendo uma classe mais ampla de gramáticas

Análise sintática descendente com retrocesso

- Pode-se, **enfraquecer as restrições** da seguinte forma:
 - Seja X um símbolo da gramática G
 - $\Psi(X) = \{Y \in V_T \mid X \rightarrow Y\alpha, \alpha \in V, X \in V\}$
 - $\Psi(X)$ contém todos os símbolos terminais Y que **derivem** de X , junto com um α



Ψ = Psi ou First

Análise sintática descendente com retrocesso

- Restrições

- Se $A \rightarrow X_1 A_1 \mid X_2 A_2 \mid \dots \mid X_n A_n$, são todas as alternativas, para o não terminal A
 - Então os conjuntos $\Psi(X_i)$ são 2 a 2 distintos
- Os X_i podem ser tanto terminais como não terminais
 - Nas restrições anteriores podiam ser só terminais

Análise sintática descendente com retrocesso

- **Restrições**

- Agora, o algoritmo de análise deve verificar a qual dos conjuntos $\Psi(X_i)$ pertence o 1º símbolo da cadeia de entrada α .
- Como os conjuntos devem ser disjuntos, haverá no máximo uma alternativa possível

Análise sintática descendente com retrocesso

- Restrições
 - Exemplo: Seja a gramática
 - $S \rightarrow AS \mid BA$
 - $A \rightarrow aB \mid C$
 - $B \rightarrow bA \mid d$
 - $C \rightarrow c$

Análise sintática descendente com retrocesso

- **Exemplo:** Seja a gramática

- $S \rightarrow AS \mid BA$

- $A \rightarrow aB \mid C$

- $B \rightarrow bA \mid d$

- $C \rightarrow c$

- **Símbolo:** $\Psi(X)$

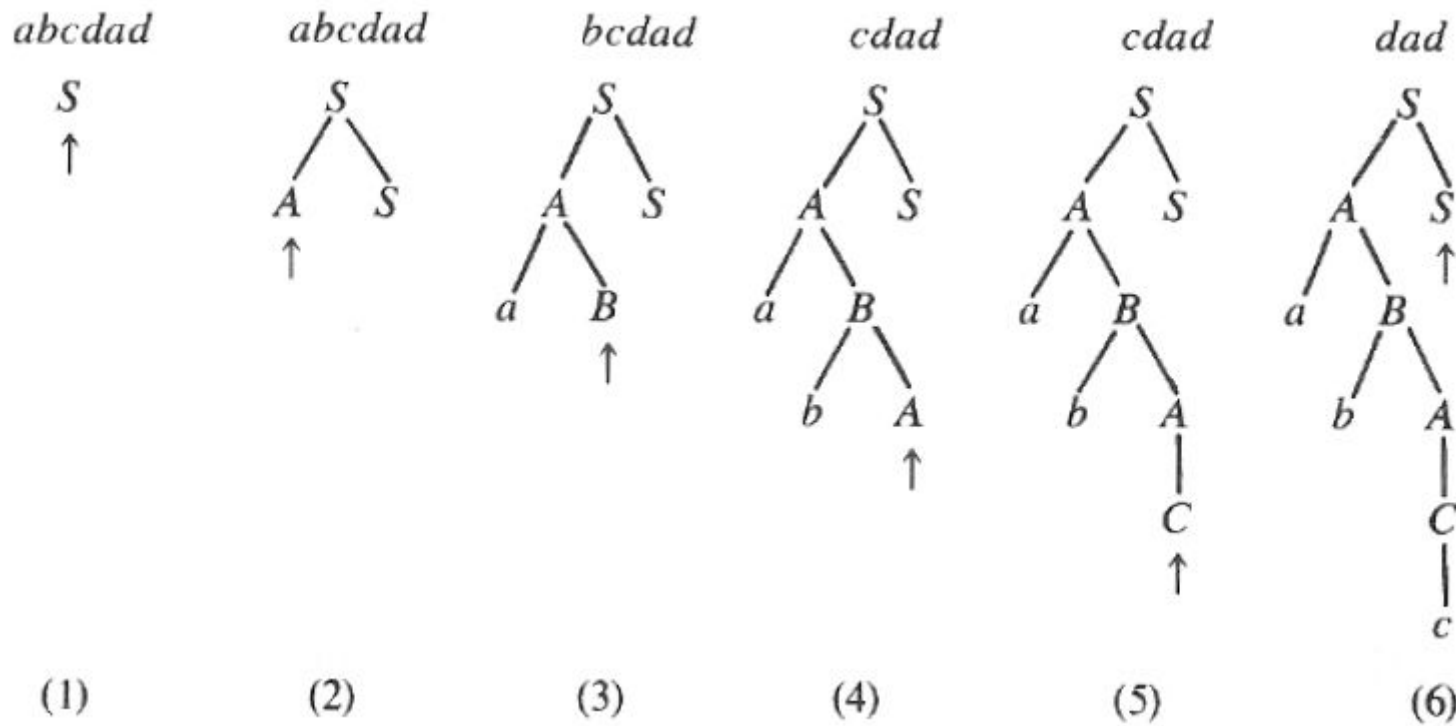
- Símbolo ψ

■ S	a,b,d,c
■ A	a,c
■ B	b,d
■ C	c

Análise sintática descendente com retrocesso

○ Exemplo:

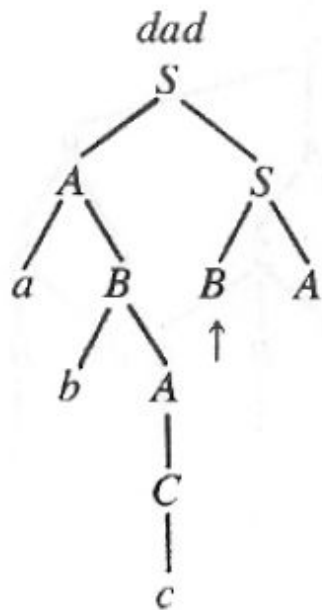
$S \rightarrow AS \mid BA$; $A \rightarrow aB \mid C$; $B \rightarrow bA \mid d$; $C \rightarrow c$



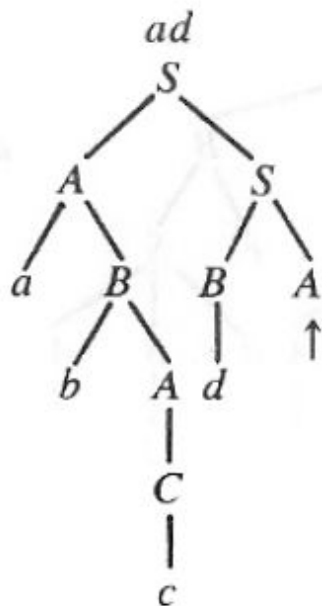
Análise sintática descendente com retrocesso

○ Exemplo:

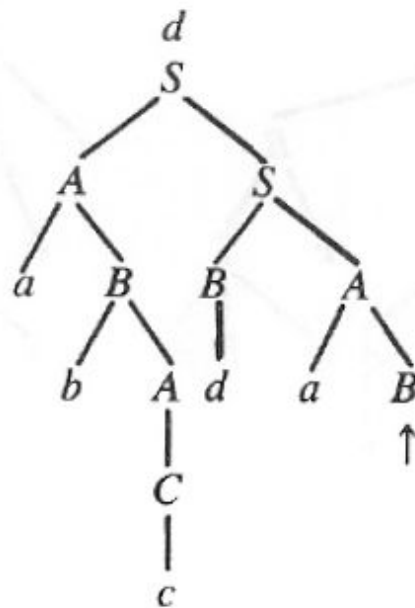
$S \rightarrow AS \mid BA$; $A \rightarrow aB \mid C$; $B \rightarrow bA \mid d$; $C \rightarrow c$



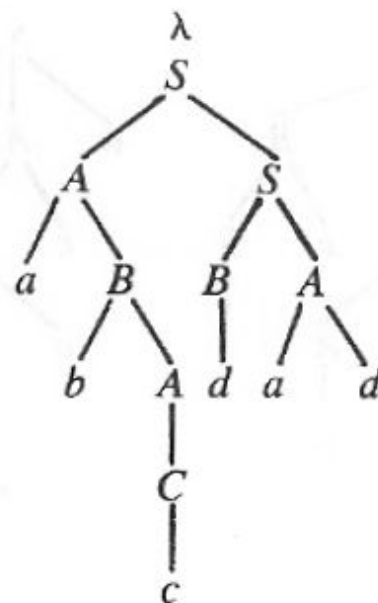
(7)



(8)



(9)



(10)

Análise sintática descendente com retrocesso

- A classe de gramáticas que satisfazem esta restrição é denominada LL(1)
- O nome vem do fato de uma cadeia poder ser analisada **da esquerda (LEFT) para a direita**, produzindo uma derivação à **esquerda (LEFT)**, verificando-se apenas 1 símbolo da cadeia de entrada
- A definição pode ser generalizada para LL(K), $K > 0$

Análise sintática descendente com retrocesso

- Para tais gramáticas podem-se obter analisadores descendentes sem retrocesso, se a escolha da produção a ser aplicada for baseada **nos k primeiros símbolos (se existirem) da cadeia corrente**
- No próximo exemplo, a gramática é do tipo LL(2), onde é necessário analisar os dois primeiros símbolos da cadeia de entrada

Análise sintática descendente com retrocesso

- Exemplo:

- $S \rightarrow aAB \mid aBA$
- $A \rightarrow b \mid cS$
- $B \rightarrow d \mid eS$

- Símbolo: $\Psi(X)$

- Símbolo ψ
 - S a,b, c, d, e
 - A b, c
 - B d, e

Análise sintática descendente com retrocesso

○ Exemplo:

$S \rightarrow aAB \mid aBA; A \rightarrow b \mid cS; B \rightarrow d \mid eS$

acadbeadb

S
↑

(1)

cadbeadb

S
/ A \ B
/ a | A | B
↑

(2)

adbeadb

S
/ A \ B
/ a | A | B
/ c | S
↑

(3)

dbeadb

S
/ A \ B
/ a | A | B
/ c | S
/ a | B | A
↑

(4)

beadb

S
/ A \ B
/ a | A | B
/ c | S
/ a | B | A
/ d | A
↑

(5)

eadb

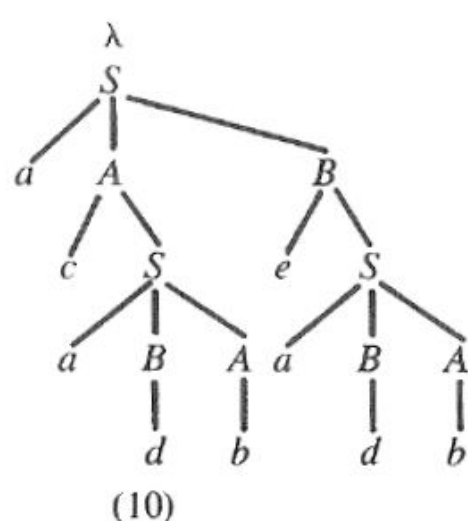
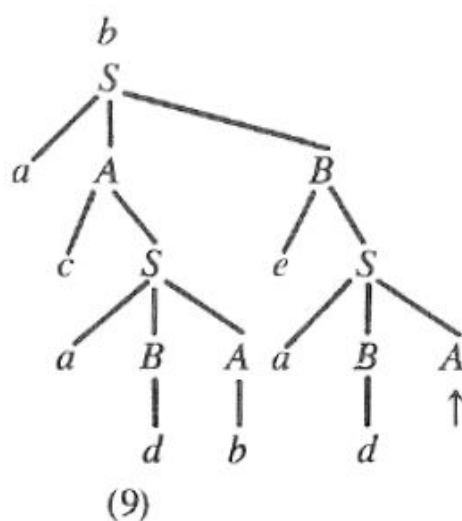
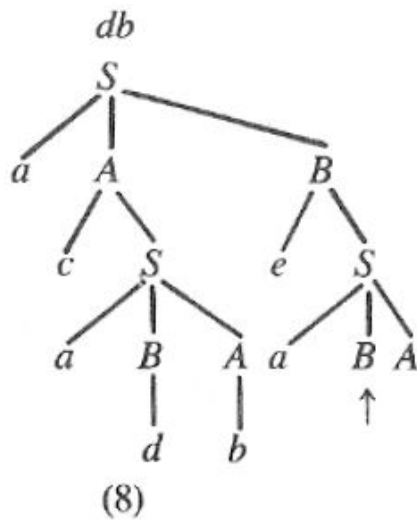
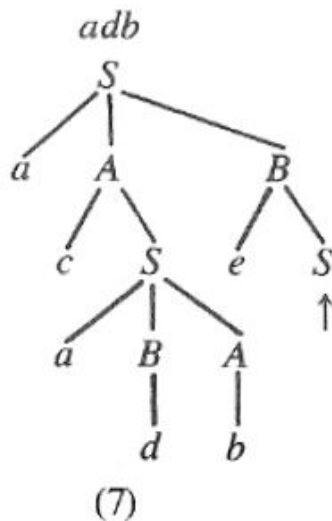
S
/ A \ B
/ a | A | B
/ c | S
/ a | B | A
/ d | b
↑

(6)

Análise sintática descendente com retrocesso

○ Exemplo:

$S \rightarrow aAB \mid aBA; A \rightarrow b \mid cS; B \rightarrow d \mid eS$



Análise sintática descendente com retrocesso

- Claramente, a primeira produção faz com que a gramática não seja do tipo LL(1)
- Entretanto, é fácil verificar que uma cadeia derivada de A começa sempre com os símbolos b ou c, e as cadeias derivadas de B começam sempre com símbolos d ou e

Análise sintática descendente com retrocesso

- Uma propriedade importante é que **toda gramática que pertence à classe LL(1) é não ambígua**
- Mesmo assim estas classes LL(K) não incluem certas gramáticas de interesse
 - **Exemplo (expressões):**
 - $E \rightarrow T + E \mid T$
 - $T \rightarrow F * T \mid F$
 - $F \rightarrow a \mid b \mid (E)$

Análise sintática descendente com retrocesso

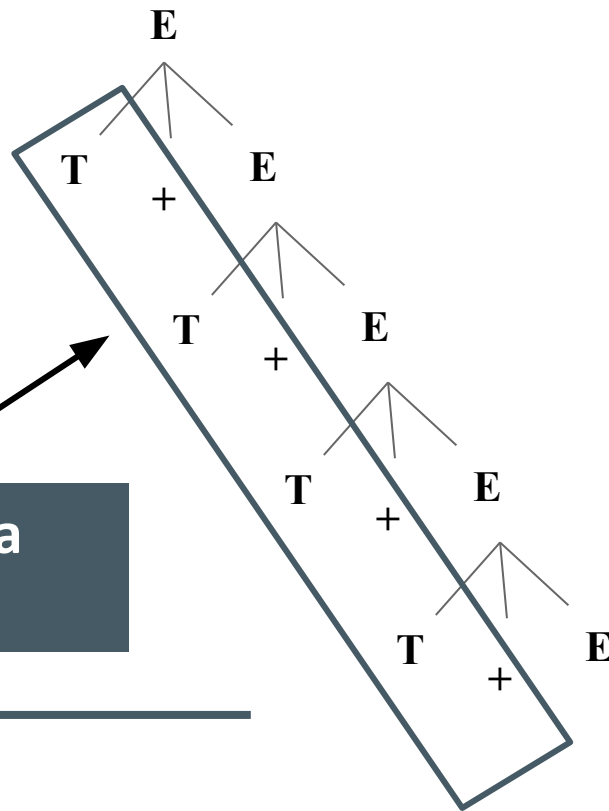
- Exemplo (expressões):

- $E \rightarrow T + E \mid T$

- $T \rightarrow F * T \mid F$

- $F \rightarrow a \mid b \mid (E)$

Recursão a
esquerda



Análise sintática descendente com retrocesso

- Esta gramática que escreve expressões, pode entrar em recursão esquerda
 - **Solução** transformar a gramática em:
 - $E \rightarrow TE'$
 - $E' \rightarrow +E \mid \lambda$
 - $T \rightarrow FT'$
 - $T' \rightarrow *T \mid \lambda$
 - $F \rightarrow a \mid b \mid (E)$

$E \rightarrow T + E \mid T$
 $T \rightarrow F * T \mid F$
 $F \rightarrow a \mid b \mid (E)$

Análise sintática descendente com retrocesso

- **Convenção**

- Produções do tipo $A \rightarrow \lambda$ serão as últimas a serem adotadas
- Alterar as gramáticas incluindo mais produções torna a análise menos eficiente
- A solução para este problema pode ser, adotando-se a **notação estendida** para as gramáticas

Análise sintática descendente - Notação estendida

- Dada a seguinte gramática livre de contexto $G = (V_N, V_T, S, P)$, suponha, que um não terminal A tenha a forma:
 - $A ::= \beta\gamma_1 \mid \beta\gamma_2 \mid \dots \mid \beta\gamma_n$
 - Com $\beta \neq \gamma$, podemos “**fatorar**” esta regra escrevendo:
 - $A ::= \beta(\gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_n)$

Análise sintática descendente - Notação estendida

- Quando for igual a λ deve ser colocada no final
- Assim $E ::= T+E \mid T$ pode ser reescrito como
 - **$E ::= T(+E/\lambda)$**

Análise sintática descendente - Notação estendida

- Por exemplo considerando a gramática:
 - $E ::= T \mid +T \mid -T \mid E+T \mid E-T$
 - $T ::= F \mid T * F \mid T / F$
 - $F ::= P \mid F ** P$
 - $P ::= i \mid (E)$

Análise sintática descendente - Notação estendida

- Pode ser reescrita como:
 - $E ::= (+ \mid - \mid \lambda)T \mid E(+T \mid -T)$
 - $T ::= F \mid T (* F \mid /F)$
 - $F ::= P \mid F (**P)$
 - $P ::= i \mid (E)$

Análise sintática descendente - Notação estendida

- Adotando a notação que $[]$ contém o λ , a gramática pode ser reescrita:
 - $E ::= [+ \mid -]T \mid E(+T \mid -T)$
 - $T ::= F \mid T(*F \mid /F)$
 - $F ::= P \mid F(**P)$
 - $P ::= i \mid (E)$

Análise sintática descendente - Notação estendida

- Surge então o problema da recursão à esquerda, que pode ser resolvido da seguinte maneira:
 - A produção $E ::= E + T \mid T$, podemos ter $\sum^* (T, T+T, T+T+T, \dots)$
 - Podemos derivar um número qualquer de símbolos T separados por $+$
 - No caso da produção citada podemos ter $E ::= T\{+T\}$

Análise sintática descendente - Notação estendida

- **Regra da Recursão**

- $N \rightarrow \alpha N \mid \beta \Rightarrow N \rightarrow \alpha^* \beta$
- $N \rightarrow N \alpha \mid \beta \Rightarrow N \rightarrow \beta \alpha^*$

- **A gramática reescrita fica:**

- $E ::= [+ \mid -]T \mid E(+T \mid -T)$
 - $T ::= F \mid T(*F \mid /F)$
 - $F ::= P \mid F(**P)$
 - $P ::= i \mid (E)$
- $E ::= [+ \mid -]T \{ +T \mid -T \}^*$
 - $T ::= F \{ *F \mid /F \}^*$
 - $F ::= P \{ **P \}^*$
 - $P ::= i \mid (E)$

Análise sintática descendente - Notação estendida

- Podemos introduzir uma nova notação, quando ocorrer uma cadeia α (que deve ocorrer pelo menos uma vez) se ela for repetida, devemos intercalá-la com β , isto é gera o conjunto $(\alpha, \alpha\beta\alpha, \alpha\beta\alpha\beta\alpha, \dots)$
- **Definiremos:**
 - $\{ \alpha // \beta \}^+$

Análise sintática descendente - Notação estendida

- Podemos finalmente representar a gramática:
 - $E ::= [+ \mid -] \{ T // (+ \mid -) \}^+$
 - $T ::= \{ F // (* \mid /) \}^+$
 - $F ::= \{ P // ** \}^+$
 - $P ::= i \mid (E)$

Análise sintática descendente - Notação estendida

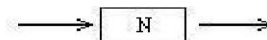
- As expressões assim obtidas são chamadas **Expressões Regulares Estendidas (ERE)** e a sua interpretação é:
 - $(\alpha) = \alpha$
 - $[\alpha] = \alpha \mid \lambda$
 - $\{\alpha\}^*$ ou $\{\alpha\} = \mid \lambda \mid \alpha \mid \alpha\alpha \mid \alpha\alpha\alpha$
 - $\{\alpha//\beta\}^+ = \alpha \{ \beta\alpha \}^*$

Análise sintática descendente - Notação estendida

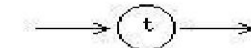
Expressão Regular Estendida
(E.R.E.)

Grafo

N (Não
Terminal)



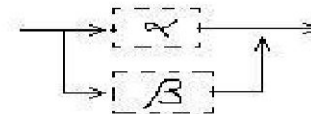
t (terminal)



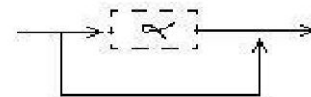
$\alpha \beta$



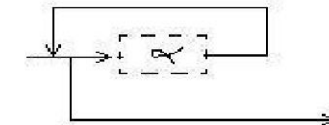
α / β



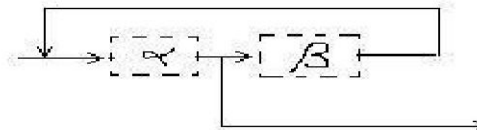
$[\alpha]$



$\{\alpha\}^*$



$\{\alpha // \beta\}^+$



Análise sintática descendente - Notação estendida

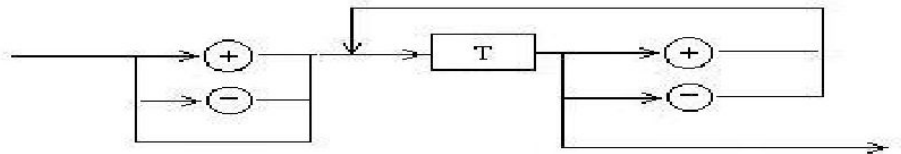
$E ::= [+ \mid -] \{ T // (+ \mid -) \}^+$

$T ::= \{ F // (* \mid /) \}^+$

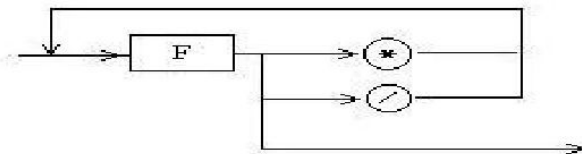
$F ::= \{ P // ** \}^+$

$P ::= E \mid (i)$

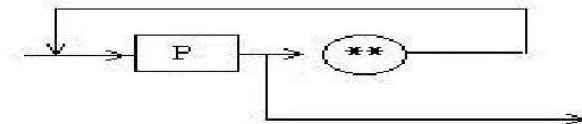
E



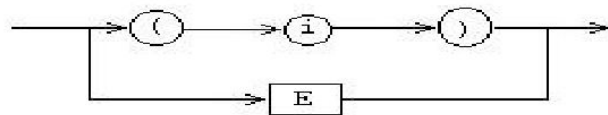
T



F



P



Análise sintática descendente - Implementação

- Proposta utilizando **estruturas de dados (PILHAS)**:
 - Adotando-se uma gramática LL(1), que não é necessário utilizar notação estendida, é proposto um processo onde cada folha da árvore será processada e empilhada, e quando o topo tiver terminal, ele é comparado com a cadeia a ser verificada

Análise sintática descendente - Implementação

- Para este caso numa implementação real as estruturas de dados devem ser estudadas
- **Exemplo:**
 - Dada a gramática:
 - $S ::= AS \mid BA$
 - $A ::= aB \mid C$
 - $B ::= bA \mid d$
 - $C ::= c$

Análise sintática descendente - Implementação

- $S ::= AS \mid BA$
- $A ::= aB \mid C$
- $B ::= bA \mid d$
- $C ::= c$
- $P = \text{Pilha}$
- $X = \text{Topo da Pilha}$

i	P	X	Cadeia Corrente
1	S	S	abcdad#
2	SA	A	abcdad #
3	SBa	a	abcdad #
2	SB	B	bcdad#
3	SAb	b	bcdad#
2	SA	A	cdad#
2	SC	C	cdad#
2	Sc	c	cdad#
1	S	S	dad#
2	AB	B	dad#
2	Ad	d	dad#
1	A	A	ad#
2	Ba	a	ad#
1	B	B	d#
1	d	d	d#
0			#

Análise sintática descendente - Implementação

procedimento ANÁLISE DESCENDENTE;

inicio

P[1]:= 'S'; i:=1; término := falso; PRÓXIMO;

repita

x:= P[i];

se "x é terminal" **então**

se x = símbolo **então**

{PRÓXIMO; i:= i-1;

se i=0 **então** término := verdadeiro}

senão ERRO

senão

se "existe uma produção $x ::= X_1 X_2 \dots X_n$ com símbolo $\in \Psi(X_1)$ " **então**

{P[i]:= 'X_n'; P[i+1]:= 'X_{n-1}'; ... P[i+n-1]:= 'X₁'; i:=i+n-1}

senão ERRO

até término

se símbolo $\neq$ '#' **então** ERRO

fim.

Análise sintática descendente - Implementação

- **Proposta utilizando procedimentos mutuamente exclusivos:**
 - Uma outra maneira simples e muito utilizada é utilizar um conjunto de procedimentos
 - Cada procedimento corresponde a um não terminal da gramática:
 - Para a gramática do exemplo anterior os procedimentos ficam:

Análise sintática descendente - Implementação

procedimento S;

inicio

caso símbolo de

'a', 'c' : {A;S};

'b', 'd' : {B;A};

outros ERRO

fim do caso

fim de S;

$S ::= AS \mid BA$

$A ::= aB \mid C$

$B ::= bA \mid d$

$C ::= c$

Análise sintática descendente - Implementação

procedimento A;

inicio

caso símbolo de

'a' : {PRÓXIMO;B};

'c' : {C};

outros ERRO

fim do caso

fim {de A};

$S ::= AS \mid BA$

$A ::= aB \mid C$

$B ::= bA \mid d$

$C ::= c$

Análise sintática descendente - Implementação

procedimento B;

inicio

caso símbolo de

'b' : {PRÓXIMO;A};

'd' : {PRÓXIMO};

outros ERRO

fim do caso

fim {de B};

$S ::= AS \mid BA$

$A ::= aB \mid C$

$B ::= bA \mid d$

$C ::= c$

Análise sintática descendente - Implementação

procedimento C;

inicio

caso símbolo de

'c' : {PRÓXIMO};

outros ERRO

fim do caso

fim {de C};

$S ::= AS \mid BA$

$A ::= aB \mid C$

$B ::= bA \mid d$

$C ::= c$

Análise sintática descendente - Implementação

inicio

PRÓXIMO; S;

se símbolo $\neq$ '#' **então** ERRO

fim.

Análise sintática descendente - Implementação

- Este tipo de processo fica facilitado para gramáticas descritas em notação estendida.
 - **Exemplo:** Dada a gramática:
 - $E ::= (+T \mid -T \mid T) \{+T \mid -T\}$
 - $T ::= F \{*F \mid /F\}$
 - $F ::= a \mid b \mid (E)$

Análise sintática descendente - Implementação

procedimento E;

inicio

caso símbolo de

'+', '-' : {PRÓXIMO;T};

outros T

fim do caso

enquanto símbolo $\in$ {'+', '-'}

faça {PRÓXIMO;T}

fim {de E};

$E ::= (+T \mid -T \mid T) \{+T \mid -T\}$

$T ::= F \{*F \mid /F\}$

$F ::= a \mid b \mid (E)$

Análise sintática descendente - Implementação

procedimento T;

inicio

F;

enquanto símbolo $\in \{^*"/^{\wedge}\}$

faça {PRÓXIMO;F}

fim {de T};

$E ::= (+T \mid -T \mid T) \{+T \mid -T\}$

$T ::= F \{*F \mid /F\}$

$F ::= a \mid b \mid (E)$

Análise sintática descendente - Implementação

procedimento F;

início

caso símbolo de

'a','b' : PRÓXIMO;

'(' : {PRÓXIMO; E ;

se símbolo = ')' então PRÓXIMO

senão ERRO};

outros ERRO

fim do caso

fim {de F};

$$E ::= (+T \mid -T \mid T) \{+T \mid -T\}$$
$$T ::= F \{*F \mid /F\}$$
$$F ::= a \mid b \mid (E)$$

Análise sintática descendente - Implementação

inicio

PRÓXIMO; E;

se símbolo $\neq$ '#' **então** ERRO

fim.